



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ

Εργαστήριο Λειτουργικών Συστημάτων Άσκηση 1 ./riddle

Βασίλης Πάντζος AM: 03121281

Ελισάβετ Αϊφαντή AM: 03122821

Challenge 0:

Εκτελώντας το riddle με την ltrace, παρατηρούμε ότι το εκτελέσιμο προσπαθεί να ανοίξει το αρχείο “.hello_there”, το οποίο δεν υπάρχει.

```
open(".hello_there", 0, 00) = -1
__fprintf_chk(0x76f3b7f6a4e0, 1, "\033[31m%s\033[0m\n", "FAIL"FAIL
) = 14
```

Για να επιτύχει η κλήση συστήματος, απλά δημιουργούμε το αρχείο με την εντολή
touch .hello_there

Challenge 1:

```
Challenge 1: 'Gatekeeper'
Hint: 'Stand guard, let noone pass'.
... I found the doors unlocked. FAIL
```

Κάνοντας strace στο εκτελέσιμο βλέπουμε το εξής κατά την κλήση της write:

```
openat(AT_FDCWD, ".hello_there", O_WRONLY) = 4
close(4) = 0
write(2, "\33[31m... I found the doors unloc...", 46... I found the doors unlocked. FAIL
) = 46
```

Η write λειτουργεί κανονικά και αυτό είναι κάτι που στο συγκεκριμένο challenge δεν θέλουμε να γίνει οπότε πρέπει απλά να αλλάξουμε τα δικαιώματα του αρχείου .hello_there έτσι ώστε να μην είναι εφικτό να γράψουμε σε αυτό. Αυτό γίνεται με την χρήση της εντολής chmod και συγκεκριμένα πρέπει να γράψουμε στο terminal:

chmod -w .hello_there

Challenge 2:

Παρατηρούμε ότι δημιουργήθηκαν 2 σήματα SIGALRM και SIGCONT με τον ίδιο signal handler. Ο σκοπός είναι να συνεχίσει η διεργασία πριν τελειώσει το timer.

```
signal(SIGALRM, 0x647bdaf0d350)
signal(SIGCONT, 0x647bdaf0d350)
alarm(10)
pause(10, 0x7ffdfa89fdc0, 0, 0x760dfc5272b7^C <no return ...>
```

Το σήμα SIGALRM καλείται μόνο από το alarm(), το οποίο δεν μπορεί να κληθεί manually. Επιλέγουμε να χρησιμοποιήσουμε το SIGCONT ώστε να κληθεί έμμεσα το alarm με την kill. Τρέχουμε το εκτελέσιμο με ltrace -f ./riddle ώστε να δούμε το pid του και σε άλλο terminal εκτελούμε:

```
kill -SIGCONT [pid του ./riddle]
```

Challenge 3:

```
Challenge 3: 'what is the answer to life the universe and everything?'
Hint:      'ltrace'.
FAIL
```

Κάνοντας ltrace βλέπουμε το εξής:

```
Challenge 3: 'what is the answer to life the universe and everything?'
)                                     = 74
__fprintf_chk(0x7b593f6044e0, 1, 0x627c442ce04b, 0x627c442d26a0Hint:      'ltrace'.
)                                     = 25
getenv("ANSWER")                      = nil
__fprintf_chk(0x7b593f6044e0, 1, 0x627c442ce06e, 0x627c442ce00cFAIL
)                                     = 14
```

Η διεργασία αναζητά το environment variable ANSWER. Η τιμή του βρίσκεται από το hint “what is the answer to life the universe and everything?” που μας παραπέμπει στο γνωστό βιβλίο “The Hitchhiker’s Guide to the Galaxy” όπου ο αριθμός αυτός είναι 42. Τρέχουμε απλά:

```
export ANSWER=42
```

Challenge 4:

Το όνομα του challenge καθώς και το μήνυμα αποτυχίας μας παραπέμπουν σε επικοινωνία μεταξύ αρχείων με την χρήση FIFO. Με ltrace:

```
open("magic_mirror", 0x2, 02437270071) = -1
```

Δημιουργήσαμε το FIFO αρχείο με την εντολή:

```
mkfifo magic_mirror
```

Σημειώνεται ότι εάν και δεν υπάρχει δεύτερη διεργασία που ανοίγει το fifo αρχείο για να επικοινωνήσει με τη riddle, η open είναι επιτυχής. Αυτό οφείλεται στο ότι έγινε ανάθεση δικαιωμάτων read και write (0x2), που επιτρέπει και στα δύο endpoints της επικοινωνίας να βρίσκονται στην ίδια διεργασία.

Challenge 5:

```
Challenge 5: 'my favourite fd is 99'
Hint:       'when I bang my head against the wall it goes: dup! dup! dup!'.
FAIL
```

Κάνοντας strace στο εκτελέσιμο βλέπουμε το εξής:

```
Challenge 5: 'my favourite fd is 99'
) = 40
write(2, "Hint:       'when I bang my h"..., 79Hint:       'when I bang my head against the wall it goes: dup! dup! dup!'.
) = 79
fcntl(99, F_GETFD) = -1 EBADF (Bad file descriptor)
write(2, "\33[31mFAIL\33[0m\n", 14FAIL
) = 14
```

Παρατηρούμε ότι η κλήση συστήματος fcntl() επιστρέφει error αφού δεν βρίσκει αρχείο με fd 99.

Η λύση φαίνεται στον παρακάτω κώδικα σε c:

```
#include <unistd.h>
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
int main(){
    int fd=open("test",O_RDWR|O_CREAT);
    printf("%d\n",fd);
    int new_fd=dup2(fd, 99);
    printf("%d\n",new_fd);
    system("./riddle");
    return 0;
}
```

Στο παραπάνω πρόγραμμα κανουμε χρήση της κλήσης συστήματος dup2() η οποία μας επιτρέπει να κάνουμε allocate ένα νέο file descriptor με τον αριθμό 99.

Challenge 6:

Με ltrace:

```
fork() = 212
fork() = 213
wait(0x7ffcaa17d7e4[212] PING!
<no return ...>
```

Δημιουργήθηκαν 2 child διεργασίες, από τις οποίες η μία τυπώνει PING!, ενώ η άλλη δεν ανταποκρίνεται. Θέλουμε η δεύτερη να τυπώνει PONG!.

Διερευνάμε τι γίνεται σε κάθε διεργασία με την strace -f ./riddle:

```
[pid 306] write(34, "\0\0\0\0", 4 <unfinished ...>
[pid 305] <... rt_sigprocmask resumed>, NULL, 8) = 0
[pid 307] <... set_robust_list resumed>) = 0
[pid 305] wait4(-1 <unfinished ...>
[pid 306] <... write resumed>) = -1 EBADF (Bad file descriptor)
[pid 307] rt_sigprocmask(SIG_SETMASK, [] <unfinished ...>
[pid 306] exit_group(1 <unfinished ...>
[pid 307] <... rt_sigprocmask resumed>, NULL, 8) = 0
[pid 306] <... exit_group resumed>) = ?
[pid 307] alarm(2) = 0
[pid 307] read(33 <unfinished ...>
[pid 306] +++ exited with 1 +++
[pid 307] <... read resumed>, 0x7ffe1ed7d514, 4) = -1 EBADF (Bad file descriptor)
```

Η μία διεργασία προσπαθεί να γράψει στο αρχείο με fd 34, ενώ η άλλη να διαβάσει το αρχείο με fd 33. Ωστόσο, τυπώνεται το error EBADF (bad file descriptor).

Με c δημιουργούμε pipe που ενώνει τα fd 33-34 ώστε να επιτρέπεται η διεργασιακή επικοινωνία. Ωστόσο, τότε λαμβάνουμε το ίδιο error, αφού τώρα η διεργασία που έγραφε στο fd 34 προσπαθεί να διαβάσει από το fd 53 και η διεργασία που διάβαζε από το fd 33, προσπαθεί να γράψει στο fd 54:

```
[pid 343] read(53 <unfinished ...>
[pid 344] <... getpid resumed>) = 344
[pid 343] <... read resumed>, 0x7ffe0cb25544, 4) = -1 EBADF (Bad file descriptor)
[pid 344] fstat(1 <unfinished ...>
[pid 343] alarm(0 <unfinished ...>
[pid 344] <... fstat resumed>, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
[pid 343] <... alarm resumed>) = 2
[pid 344] getrandom( <unfinished ...>
[pid 343] exit_group(1 <unfinished ...>
[pid 344] <... getrandom resumed>"\xa9\x72\xb0\x72\x6f\xb6\x5b\x07", 8, GRND_NONBLOCK) = 8
[pid 343] <... exit_group resumed>) = ?
[pid 344] brk(NULL) = 0x5ce9f7d46000
[pid 343] +++ exited with 1 +++
[pid 344] brk(0x5ce9f7d67000 <unfinished ...>
[pid 342] <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 1}], 0, NULL) = 343
[pid 344] <... brk resumed>) = 0x5ce9f7d67000
[pid 342] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=343, si_uid=1000, si_status=1, si_etime=0, si_stime=0} ---
[pid 342] wait4(-1 <unfinished ...>
[pid 344] write(1, "[344] PONG!\n", 12[344] PONG!
) = 12
[pid 344] getpid() = 344
[pid 344] write(1, "[344] PING!\n", 12[344] PING!
) = 12
[pid 344] write(54, "\1\0\0\0", 4) = -1 EBADF (Bad file descriptor)
[pid 344] exit_group(1) = ?
[pid 344] +++ exited with 1 +++
```

Δημιουργήσαμε έτσι και δεύτερο pipe για τα fd 53-54:

```
#include <unistd.h>

int main(){

    int fd1[2]; //33-34 pair
    int fd2[2]; //53-54 pair

    pipe(fd1);
    pipe(fd2);

    dup2(fd1[0],33);
    dup2(fd1[1],34);

    dup2(fd2[0],53);
    dup2(fd2[1],54);

    execl("./riddle", "riddle", NULL);
    return 0;
}
```

Οπότε τώρα οι δύο διεργασίες μπορούν να επικοινωνούν με 2 pipes αφού καθεμία έχει read και write end.

Challenge 7:

```
Challenge 7: 'What's in a name?'  
Hint:      'A rose, by any other name...'.  
FAIL
```

Κάνοντας strace:

```
Challenge 7: 'What's in a name?'  
) = 36  
write(2, "Hint:      'A rose, by any o"... , 47Hint:      'A rose, by any other name...'.  
) = 47  
lstat(".hello_there", {st_mode=S_IFREG|0444, st_size=0, ...}) = 0  
lstat(".hey_there", 0x7fff39c48b30) = -1 ENOENT (No such file or directory)  
write(2, "\33[31mFAIL\33[0m\n", 14FAIL  
) = 14  
write(2, "\nChallenge 8: 'Big Data'\n", 27
```

Παρατηρούμε πως πρέπει να κάνουμε ένα hard link μεταξύ του .hello_there με το .hey_there. Αυτό γίνεται με χρήση της εντολής ln που κάνει hard links by default.

Συγκεκριμένα:

ln .hello_there .hey_there

Challenge 8:

Το πρόγραμμα προσπαθεί επανειλημμένα να ανοίξει το αρχείο bf00 αποτυχημένα.

```
open("bf00", 0, 04770240071) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
open("bf00", 0, 00) = -1
```

Έγινε πρώτη προσπάθεια με την touch bf00. Τότε, το πρόγραμμα ενώ έβρισκε το αρχείο, διάβαζε πολλαπλές φορές το ίδιο offset στο αρχείο, ψάχνοντας εάν υπάρχουν 16 χαρακτήρες. Το hint “data must be present and whole” υποδεικνύει ότι πρέπει το αρχείο να έχει μέγεθος όσο το offset+16 bytes και να έχουν όλα κάποια δεδομένα.

```
open("bf00", 0, 00) = 4
lseek(4, 1073741824, SEEK_SET) = 1073741824
read(4, "", 16) = 0
__printf_chk(1, "%s", "X") = 1
fflush(0x7c2002f495c0X) = 0
close(4) = 0
```

Για να γίνει αυτό, σε πρόγραμμα c δημιουργήσαμε αρχείο bf00, με την lseek θέσαμε το ζητούμενο pointer το αρχείο και με την write το γεμίσαμε με “1...1” (16 byte string). Παρατηρήσαμε ότι τότε το πρόγραμμα ζητάει το ίδιο για 10 αρχεία bf00, bf01, ..., bf09. Οπότε κάναμε το ίδιο για όλα τα 10 αρχεία με ένα for loop.

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(){
    int k = 0;
    char filename[5];
    while (k<10){
        sprintf(filename, "bf0%01i", k);
        int fd = open(filename, O_RDWR | O_CREAT, 0644);
        ftruncate(fd, 1073741840);
        lseek(fd, 1073741824, SEEK_SET);
        write(fd, "1111111111111111", 16);
        close(fd);
        k++;
    }
    execlp("./riddle", "riddle", NULL);
    return 0;
}
```

```
open("bf09", 0, 00) = 4
lseek(4, 1073741824, SEEK_SET) = 1073741824
read(4, "1111111111111111", 16) = 16
__printf_chk(1, "%s", ".") = 1
fflush(0x7dddb2dcb5c0.) = 0
close(4) = 0
__fprintf_chk(0x7dddb2dcb4e0, 1, "\\033[32m%s\\033[0m\\n", "SUCCESS"SUCCESS) = 17
```


Challenge 9:

```
Challenge 9: 'Connect'
Hint:      'Let me whisper in your ear'.
I am trying to contact you...\n...but you don't seem to listen.\nFAIL
```

Κάνοντας strace:

```
Challenge 9: 'Connect'
) = 26
write(2, "Hint:      'Let me whisper i"...45Hint:      'Let me whisper in your ear'.
) = 45
socket(AF_INET, SOCK_STREAM, IPPROTO_IP) = 4
write(2, "I am trying to contact you...\n", 31I am trying to contact you...\n) = 31
connect(4, {sa_family=AF_INET, sin_port=htons(49842), sin_addr=inet_addr("127.0.0.1")}, 16) = -1 ECONNREFUSED (Connection refused)
write(2, "...but you don't seem to listen."..., 34...but you don't seem to listen.\n) = 34
write(2, "\33[31mFAIL\33[0m\n", 14FAIL
) = 14
```

Παρατηρούμε πώς αποτυγχάνει η κλήση συστήματος connect στο port 4984.
Ενώ έτρεχε το πρόγραμμα σε άλλο terminal:
nc -l 49842

```
kill -18 $pid
(base) vagg@Ubuntu:~/Desktop/report/riddle-20250930-0$ nc -l 49842
How much is 32842 + 1? █
```

Με την σημαία -l δεν δημιουργούμε κάποια σύνδεση αλλά αναμένουμε να γίνει κάποιο αιτήμα(απο την connect).

Challenge 10:

Το πρόγραμμα ανοίγει ένα αρχείο `secret_number` και ύστερα το κάνει `unlink`, διαγράφοντας το `hard link` του. Το αρχείο παύει να υπάρχει στο σύστημα αρχείων, ωστόσο μπορεί ακόμα να γίνει αναφορά στο `file descriptor` του όσο εκτελείται η διεργασία. Για να έχουμε πρόσβαση στα δεδομένα του, δημιουργήσαμε αρχείο `secret_number_` και το κάναμε `link` με το `secret_number` πριν ξεκινήσει η εκτέλεση του προγράμματος.

```
open("secret_number", 0x242, 0600) = 4
unlink("secret_number")           = 0
sysconf(_SC_PAGE_SIZE)           = 4096
malloc(4096)                      = 0x57c8396152a0
write(4, "The number I am thinking of right...", 4096) = 4096
mmap(nil, 4096, 0b11, 0x1, 4, 0) = 0x750a6885e000
close(4)                          = 0
```

touch secret_number

In secret_number not_secret_number

Σε ξεχωριστά `terminal`, εκτελούμε το `riddle` και τυπώνουμε το `secret_number_`. Το αποτέλεσμα είναι ένας δήμιος δεκαεξαδικός αριθμός τον οποίο δίνουμε σαν `input` στο `riddle`.

Challenge 11:

Η προσέγγιση στο `challenge 10` εδώ αποτυγχάνει. Δοκιμάσαμε αρχικά να πάρουμε το `memory dump` με `gdb`, ωστόσο δεν επιστρέφει το `offset` στη μνήμη της διεργασίας.

Λόγω των περιορισμένων δικαιωμάτων του `gdb`, χρησιμοποιήσαμε την εντολή `dd` στο `bash` με δικαιώματα `root`. Τρέξαμε το πρόγραμμα σε άλλο `terminal` με `ltrace -f` για να δούμε το `pid` και τη διεύθυνση που επιστρέφει το `mmap`.

```
└─$ sudo dd if=/proc/725/mem skip=$((0x7b9aa73e1000)) bs=1 count=4096
dd: /proc/725/mem: cannot skip to specified offset
The number I am thinking of right now is [uppercase matters]: C7755A82
4096+0 records in
4096+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 0.0479643 s, 85.4 kB/s
```

Challenge 12:

Το πρόγραμμα σε κάθε εκτέλεση ζητάει να βρει ένα τυχαίο γράμμα σε κάποια τυχαία θέση μνήμης του.

```
[pid 770] mkstemp(0x7ffc9503f950, 0x7ffc9503f800, 0, 0) = 4
[pid 770] ftruncate(4, 4096) = 0
[pid 770] mmap(nil, 4096, 0b11, 0x1, 4, 0) = 0x7f5c441e1000
[pid 770] memset(0x7f5c441e1000, '\0', 4096) = 0x7f5c441e1000
[pid 770] rand() = 806106083
[pid 770] __fprintf_chk(0x7f5c441c74e0, 1, "%s '%c' at %p\n", "I want to find the char", 'D', 0x7f5c441e106f) = 46
[pid 770] I want to find the char 'D' at 0x7f5c441e106f
```

Ομοίως, με ltrace -f πήραμε το pid, το ζητούμενο γράμμα και τη διεύθυνση στη μνήμη και σε άλλο terminal τρέξαμε:

```
echo -n D | sudo dd of=/proc/770/mem seek=$((0x7f5c441e106f)) bs=1
```

Challenge 13:

```
Challenge 13: 'Bus error'
Hint: 'Memquake! Don't lose the pages beneath your something something'
FAIL
```

Κάνοντας strace:

```
Challenge 13: 'Bus error'
) = 28
write(2, "Hint: 'Memquake! Don't ...", 82Hint: 'Memquake! Don't lose the pages beneath your something something'.
) = 82
openat(AT_FDCWD, ".hello_there", O_RDWR|O_CREAT, 0600) = -1 EACCES (Permission denied)
write(2, "\33[31mFAIL\33[0m\n", 14) = 14
write(2, "\nNext challenge locked. Complete...", 50
Next challenge locked. Complete more challenges.
) = 50
exit_group(1) = ?
```

Αρχικά κάνουμε `chmod +w .hello_there` για να προσθέσουμε το write permission.

Στη συνέχεια έχουμε το εξής:

```
Challenge 13: 'Bus error'
) = 28
write(2, "Hint: 'Memquake! Don't ...", 82Hint: 'Memquake! Don't lose the pages beneath your something something'.
) = 82
openat(AT_FDCWD, ".hello_there", O_RDWR|O_CREAT, 0600) = 4
ftruncate(4, 32768) = 0
mmap(NULL, 32768, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7bafb0cbb000
ftruncate(4, 16384) = 0
read(0, 131
"1", 1) = 1
--- SIGBUS {si_signo=SIGBUS, si_code=BUS_ADRERR, si_addr=0x7bafb0cbf000} ---
+++ killed by SIGBUS (core dumped) +++
Bus error (core dumped)
```

Πρόκειται για ένα bus error. Σε ένα διαφορετικό terminal κάνουμε χρήση της εντολής

`truncate -s 32768 .hello_there ,`

η οποία θα μας κάνει shrink το αρχείο `.hello_there`.

Challenge 14

Το πρόγραμμα ζητάει να του αποδοθεί pid 32767.

```
[pid 894] __fprintf_chk(0x7046713074e0, 1, "Hint:          '%s'.\n", "Are you 32767?
If not, reincarna"...Hint:          'Are you 32767? If not, reincarnate!'.
) = 54
[pid 894] getenv("I_NEED_TECH_HINTS") = nil
[pid 894] getpid() = 894
[pid 894] __fprintf_chk(0x7046713074e0, 1, "\033[31m%s\033[0m\n", "I don't like my P
ID, get me anot"...I don't like my PID, get me another one. FAIL
) = 55
```

Επειδή δεν είναι δυνατό να επιλέξουμε το pid του εκτελέσιμου, κάναμε `fork()` επανειλημμένα έως ότου να παραχθεί διεργασία με pid 32767. Τότε με την `execl` η διεργασία αυτή εκτελεί το `riddle`.

```
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <stdio.h>

int main(){
    pid_t pid;
    while (1){
        if (pid==32767){
            execl("./riddle", "riddle", NULL);
            return 0;
        }
        pid = fork();
        if (pid==0) exit(0);
    }
}
```

Challenge 15-16

Με την ltrace παρατηρούμε ότι το riddle προσπαθεί να ανοίξει το αρχείο tier2.so και αποτυχαίνει. Έτσι δημιουργήσαμε αρχείο tier2.so, με ένα απλό constructor.

Στη συνέχεια, στο challenge 17 η διεργασία θέλει να δει το pid της στη διεύθυνση 0x6042000.

```
I feel like looking at address 0x6042000  
I wanted to find my PID inside a long int at this address. FAIL
```

Παρατηρήσαμε ότι φορτώνει εκ νέου το tier2.so. Αφού μοιράζονται το ίδιο pid, δημιουργήσαμε στο tier2 μια συνάρτηση setup_tier2 η οποία αποθηκεύει το pid του στη ζητούμενη διεύθυνση.

```
#define _GNU_SOURCE  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/mman.h>  
  
void setup_tier2(void){  
    int pid = (int)getpid();  
    void *addr = (void*)0x6042000;  
    *((int*)addr) = pid;  
}  
  
__attribute__((constructor))  
static void tier2_constructor(void) {  
    void *p = mmap((void*)0x6042000, 4096, PROT_READ | PROT_WRITE, MAP_PRIVATE |  
    MAP_ANONYMOUS | MAP_FIXED, -1, 0);  
    setup_tier2();  
}
```

Challenge 17

Παρατηρούμε ότι το εκτελέσιμο επιχειρεί να καλέσει την authorize_tier3. Αυτό αποτυχαίνει ωστόσο καθώς γίνεται λάθος χειρισμός των handles της βιβλιοθήκης tier3.so.

Αρχικά, επιχειρήσαμε να ανοίξουμε τη βιβλιοθήκη στον constructor της tier2 με:

```
void* tier3_handler = dlopen("tier3.so", RTLD_GLOBAL |  
RTLD_NODELETE);
```

Αφού αυτό δεν δούλεψε, δοκιμάσαμε επίσης να καλέσουμε στην tier2 την authorize:

```
dlsym(tier3_handler, "authorize_tier3");
```

Το οποίο επίσης δεν λειτούργησε. Βλέπουμε τα σύμβολα της tier3 με nm -d tier3.so. Υπάρχει μια συνάρτηση unlock_tier3, η οποία πάντα δημιουργεί segmentation fault. Με τη χρήση decompiler, ανακαλύψαμε πως προσπαθεί πάντα να κάνει override τη θέση μνήμης 0xFFFFFFFFFFFFFFFF.

Για να κάνουμε bypass το πρόβλημα αυτό, δημιουργήσαμε δική μας tier3.so που πάντα είναι επιτυχής για κάθε κλήση των authorize και unlock. Μετανομάσαμε την παλιά tier3.so ώστε να καλείται η δική μας.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdint.h>

__attribute__((constructor))

int authorize_tier3(void) {
    return 0;
}

int unlock_tier3(void) {
    return 0;
}
```


Challenge 18:

Λύσαμε το challenge κάνοντας χρήση του radare2. Ανοίγοντας το εκτελέσιμο με `r2 -AA ./riddle` και κάνοντας `iz` βρίσκουμε τη διεύθυνση της συμβολοσειράς του prompt. Στη συνέχεια με `axt` βλέπουμε που στο πρόγραμμα γίνεται το `puts`, συγκεκριμένα στην διεύθυνση `0x5252`.

```
[x] Analyze function calls (aac)

[x] Analyze len bytes of instructions for references (aar)
[x] Finding and parsing C++ vtables (avrr)
[x] Type matching analysis for all functions (aافت)
[x] Propagate noreturn information (aanr)
[x] Finding function preludes
[x] Enable constraint types analysis for variables
[0x00002380]> iz~"What's the password"
[0x00002380]> iz~password
29 0x000062bd 0x000062bd 20 21 .rodata ascii What's the password?
[0x00002380]> axt 0x000062bd
(nofunc) 0x5252 [DATA] lea rdi, str.Whats_the_password_
[0x00002380]> pdf @ 0x5252

p: Cannot find function at 0x00005252
[0x00002380]>
```

Βλέπουμε ότι δεν μπορεί το r2 μία συνάρτηση σε αυτή την διεύθυνση οπότε θα κάνουμε χρήση του `af @ 0x5252` που κάνει create ένα function για να το κάνει analyze το r2. Στη συνέχεια κάνουμε print με `pdf @ 0x5252`.

Στο αρκετά μεγάλο output βλέπουμε αυτό:

```
movabs rcx, 0x9f4bfb940868bd1c
movabs rdx, 0x4cfd9b46ad2a3004
xor rcx, qword [rax + 8]
xor rdx, qword [rax]
or rcx, rdx
je success
```

Αυτό σημαίνει: ο buffer που παράγει το πρόγραμμα (μετά την επεξεργασία) πρέπει να ισούται με αυτές τις δύο σταθερές (πρώτα 8 bytes με `rdx`, επόμενα 8 bytes με `rcx`).

Όμως μεταξύ fgets και του συγκριτικού ελέγχου υπάρχει η κλήση fcn 00003d80. Άρα ο έλεγχος δεν συγκρίνει την πρώτη πρώτη συμβολοσειρά που πληκτρολογούμε, αλλά το αποτέλεσμα αυτής της συνάρτησης ,δηλαδή η είσοδος μετασχηματίζεται πριν γίνει η σύγκριση.

Κάνοντας pdf @ 0x3d80 βλέπουμε πως από την αποσυναρμολόγηση για κάθε εισερχόμενο byte εκτελούνται οι εντολές:

```
t = s xor x
t = t xor 1
s = rotate_right(t, 1)
out[i] = s
```

Άρα το out[i] εξαρτάται από το προηγούμενο s και το τρέχον x[i].

Αντιστρέφοντας την διαδικασία :

```
s_prev = 0x6d
x = ( t xor s_prev ) xor 1
s_prev = out[i]
```

Αφού ξέρουμε ποιο out θέλει το πρόγραμμα(οι 2 σταθερές), μπορούμε βήμα-βήμα να υπολογίσουμε τα x[i]. Αυτό ακριβώς κάναμε και ως αποτέλεσμα πήραμε τα ASCII bytes:

64 65 65 70 20 70 61 64 75 66 6c 79 20 62 6c 75

που ως ASCII είναι:

deep paduflly blu

Πηγαίνοντας στο εργαστήριο είδαμε την υπόδειξη ότι ο κωδικός τελικά είναι deep paduflly blue.

Αφού κάναμε input τον κωδικό, η προσπάθεια εκτέλεσης του riddle έχει ως αποτέλεσμα:

```
➤$ ./riddle
-bash: ./riddle: cannot execute binary file: Exec format error
```

Παράγοντας το hexdump του αρχείου με xxd, παρατηρούμε ότι έχει IMP header, αντί για elf.

Για να γίνει πάλι εκτελέσιμο, απλά αλλάξαμε τα magic bytes σε elf με την παρακάτω εντολή bash:

```
printf '\x7FELF' | dd of=riddle bs=1 seek=0 count=4 conv=notrunc
```

Ως αποτέλεσμα, εμφανίζεται το παρακάτω goblin και έχουμε φτάσει στο τέλος του riddle.

└─\$./riddle



jgs

Are you a Wizard?